

Wrench Documentation

Wrench is an educational project designed to explore different types of processor architectures. This documentation explains how to write assembly code and configure simulations in a way that works across all supported Instruction Set Architectures (ISAs).

Table of Contents

- [Wrench Documentation](#)
 - [File Types and Formats](#)
 - [Supported Architectures](#)
 - [Assembly Program Structure](#)
 - [Comments](#)
 - [Labels](#)
 - [Data Section](#)
 - [Text Section](#)
 - [Setting Section Addresses](#)
 - [Configuration Files](#)
 - [Configuration File Structure](#)
 - [Configuration Options](#)
 - `limit`
 - `memory_size`
 - `memory_mapped_io`
 - `reports`
 - `name`
 - `slice`
 - `view`
 - `assert`

File Types and Formats

Wrench uses several types of files:

- **Assembly files** (`.s` extension) - Source code written in ISA-specific assembly language
- **Configuration files** (`.yaml` extension) - YAML files that control simulation parameters
- **Output** - Results and reports are printed to stdout (standard output)

For examples of fully working programs, see:

- [Example directory](#)
- [Test golden directory](#)

Supported Architectures

Wrench currently supports the following architectures, each with its own instruction set and semantics:

- [Acc32](#) - Accumulator-based 32-bit architecture
- [F32a](#) - Stack-based 32-bit architecture
- [M68k](#) - Motorola 68000-inspired architecture
- [RISC-IV](#) - RISC-V-inspired 32-bit architecture
- [VLIW-IV](#) - RISC-V-inspired VLIW 32-bit architecture

Assembly Program Structure

Assembly programs in Wrench follow a common structure regardless of the specific ISA being used. This section describes the general components that all assembly programs should have.

Assembly programs consist of **data** (containing variables, constants, and other data) and **text** (instructions) sections that can be defined in any order and multiple times throughout the file. The assembler will merge all sections of the same type into a single program. All assembly programs should be saved with a `.s` file extension.

Comments

Comments are used to add explanatory notes to your code. The comment character may vary depending on the ISA:

- Most architectures use `;` for comments
- Some architectures may use other characters (check the specific ISA documentation)

Everything after the comment character on the same line is ignored by the assembler:

```
load_addr input_addr      ; This is a comment explaining the
instruction
```

Labels

Labels are used to mark specific locations in your code or data section. They serve as symbolic names for memory addresses and make your code more readable and maintainable.

- Labels are defined by a name followed by a colon `:`
- All labels defined in the program must be unique
- Labels can be used in instructions or data declarations to reference memory locations

```
loop_start:      ; This defines a label called "loop_start"
    instruction
    instruction
    jmp loop_start ; Jump back to the label
```

Data Section

The data section is where you define variables, constants, and other data used by your program. It starts with the `.data` directive.

A data line in the data section typically consists of a label, a directive, and one or more values:

```
label:    directive    value(s)
```

- **label:** An identifier followed by a colon `:` that marks the memory location
- **directive:** Specifies the type and size of data being defined
- **value(s):** The actual data to be stored (multiple values can be separated by commas)

Common data directives include:

- `.word` - Define word-sized data (typically 32 bits)
- `.byte` - Define byte-sized data (8 bits)

Examples:

```
.data
counter:      .word 0          ; Define a word-sized variable initialized
to 0
message:      .byte 'H', 'e', 'l', 'l', 'o', 0 ; Null-terminated string
large_number: .word 0xFFFFFFFF ; Hexadecimal value
neg_one:      .word -1         ; Negative value
c_string:     .byte 'Hello, World!\0' ; C-style string with null
terminator
pascal_string: .byte 13, 'Hello, World!' ; Pascal-style string with
length prefix
```

Text Section

The text section contains the executable code (instructions) of your program. It starts with the `.text` directive.

The specific instructions available depend on the ISA you're using, but the general structure is:

```
.text
label:                ; Optional label
    instruction operands ; Instruction with its operands
    instruction operands
    ...
```

For ISA-specific details (instructions, registers, etc.), refer to the respective documentation:

- [Acc32](#) - Accumulator-based architecture
- [RISC-IV](#) - RISC-V-inspired architecture
- [F32a](#) - Stack-based architecture
- [M68k](#) - Motorola 68000-inspired architecture
- [VLIW-IV](#) - RISC-V-inspired VLIW 32-bit architecture

If you can't find certain details in the documentation, the [source code](#) contains the most up-to-date information.

Setting Section Addresses

The `.org` directive sets the starting address for a section. This is useful when you need to place code or data at specific memory addresses:

```
.data
.org 0x1000      ; Data section starts at address 0x1000
value: .word 42

.text
.org 0x2000      ; Text section starts at address 0x2000
_start:
    load_ind value
```

If `.org` is not specified, the assembler will automatically assign addresses starting from 0x0000 for the first section and place other sections sequentially.

Configuration Files

Wrench uses YAML configuration files (`.yaml` extension) to control simulation parameters. These files allow you to:

1. Set execution limits
2. Define input/output streams
3. Configure memory
4. Generate and check reports

Configuration File Structure

A configuration file typically contains the following sections:

```
# Execution limits
limit: 1000           # Maximum instructions to execute
memory_size: 8192     # Memory size in bytes

# I/O configuration
memory_mapped_io:
  0x80: [5, 6, 7]     # Input values at address 0x80
  0x84: []             # Output-only port at address 0x84

# Reports configuration
reports:
- name: "Execution trace"
  slice: all
  view: |
    {pc}: {instruction:next} {pc:label}

- name: "Result verification"
  slice: last
  view: |
    numio[0x84]: {io:0x84:dec}
  assert: |
    numio[0x84]: [] >>> [120]
```

Configuration Options

limit

- **Type:** Integer
- **Description:** Specifies the maximum number of instructions the simulation can execute. If the simulation exceeds this limit, it will be terminated.
- **CLI Override:** `--instruction-limit LIMIT` sets the upper limit for that option.
- **Example:**

```
limit: 40
```

memory_size

- **Type:** Integer
- **Description:** Specifies the memory size in bytes.
- **CLI Override:** `--memory-limit SIZE` sets the upper limit for that option.
- **Example:**

```
memory_size: 40
```

memory_mapped_io

- **Type:** Map of decimal or hexadecimal addresses to lists of inputs
- **Description:** Defines the memory-mapped IO streams for the simulation. Each key is a memory address, and the value is a list of inputs that should be fed into the simulation at that address. To define an output port only, leave the list empty.
- **Example:**

```
memory_mapped_io:
  0x80: [5] # Input port at address 0x80 with value 5
  132: []   # Output-only port at address 132 (decimal)
```

reports

- **Type:** List of report configurations
- **Description:** Specifies the reports to generate during the simulation. Each report configuration includes settings such as the name, slice, filter, inspector, and assertions.
- **Example:**

```
reports:
- name: Step-by-step log
  slice: all
  view: |
    {pc}: {instruction:next} {pc:label}
```

Each report configuration can include the following fields:

name

- **Type:** String (optional)
- **Description:** The name of the report, used as a header in the generated output.
- **Example:**

```
name: Step-by-step log
```

slice

- **Type:** String or List
- **Description:** Specifies which part of the simulation records should be included in the report. Possible values are:
 - "all" : Include all records.
 - ["head", n] : Include the first n records.
 - ["tail", n] : Include the last n records.
 - "last" : Include only the last record.
- **Example:**

```
slice: all
```

view

- **Type:** String (template)
- **Description:** Text template to print log records. In the template, you can use state view expressions in curly brackets.
- **Example:** program counter: {pc}

General state view expressions implemented for all ISAs:

- `pc:dec` , `pc:hex` -- Print program counter in decimal or hexadecimal format.
- `pc:label` -- Print `@label-name` if current program counter is assigned with a label.
- `instruction:next` -- Print the next instruction to execute, or `-` when the machine has halted.
- `instruction:prev` -- Print the instruction that was just executed, or `-` for the initial state (before any instruction has run).
- `memory:<a>:<b>` -- Print memory dump between addresses `<a>` and `<b>` .
- `io:<a>:dec` , `io:<a>:sym` , `io:<a>:hex` -- Print input-output stream state for the specific address in decimal, symbol, or hexadecimal format. Printable char codes: [32, 126]. Also `\0` , `\n` will be printed as is. Other non-printable characters will be replaced with `?` .

Execution and memory statistics. These are typically used with `slice: last` to emit one final summary line; the values are the totals for the whole run.

- `sim:instruction-count` -- Number of instructions executed so far. With `slice: all` it shows the running step counter (0, 1, 2, ...); with `slice: last` it shows the total for the run.
- `layout:sections-size` -- Sum of byte sizes of all sections (no gaps from `.org`).
- `layout:text-sections-size` , `layout:data-sections-size` -- Same, split by section kind.
- `layout:text-ranges` -- Address ranges of all declared `.text` sections (e.g. `0x0..0x27` , `0x40..0x5b`). Hex by default; `:dec` / `:hex` suffix to override.
- `layout:data-ranges` -- Address ranges of all declared `.data` sections. Same `:dec` / `:hex` suffix.
- `mem:instr-ranges` -- Address ranges of instruction fetches at runtime, rendered as comma-separated `lo..hi` clusters (e.g. `0x0..0x4b` , `0x8c..0xbf`). Addresses are hex by default; append `:dec` or `:hex` to override (e.g. `{mem:instr-ranges:dec} -> 0..75, 140..191`).
- `mem:data-ranges` -- Address ranges of data reads and writes (merged into one set). Same `:dec` / `:hex` suffix.
- `mem:io-ranges` -- Address ranges of memory-mapped IO accesses. Same `:dec` / `:hex` suffix.

- `memory:table` -- Multi-line address-space table that puts the above together: one row per declared `text` / `data` section, one per IO cluster, and `x` rows for free regions (split at access boundaries, so the stack gets its own row). Columns: `Kind` , `Range` , `Bytes` , `Accessed` , `Coverage` . The `Bytes` column sums to `memory_size` as a built-in sanity check.
- `isa-specific` -- ISA-specific summary block. Each architecture fills it with its own stat lines (e.g. `vliw:load-percent` + `vliw:avg-load` + `vliw:bundles-by-load` on `vliw-iv`; `f32a:data-stack-max` + `f32a:return-stack-max` on `f32a`); architectures without any render an empty block. Lets one report template stay uniform across ISAs without emitting `[unknown-view]` for the variables that don't apply.

Example -- print a stats summary after the simulation finishes:

```
reports:
- name: stats
  slice: last
  view: |
    sim:instruction-count:      {sim:instruction-count}
    layout:sections-size:      {layout:sections-size}
    layout:text-sections-size: {layout:text-sections-size}
    layout:data-sections-size: {layout:data-sections-size}
    mem:instr-ranges:          {mem:instr-ranges}
    mem:data-ranges:           {mem:data-ranges}
    mem:io-ranges:             {mem:io-ranges}
```

Comparing `layout:*-size` with `mem:*-ranges` reveals which declared bytes the program actually touched and which addresses it accessed outside any declared section -- the stack region is a typical example.

For the same picture in one shot, use `{memory:table}` -- it walks the whole address space and renders one row per section / IO cluster / free span. A typical output looks like:

Kind	Range	Bytes	Accessed	Coverage
text	0x000..0x027	40	0x000..0x027	100%
data	0x028..0x02b	4	0x028..0x02b	100%
x	0x02c..0x07f	84	-	0%
io	0x080..0x087	8	0x080..0x087	100%
text	0x090..0x0e3	84	0x090..0x0e3	100%
data	0x0e4..0x0f9	22	0x0e4..0x0ee, 0x0f5..0x0f9	72% <-
partial: middle bytes untouched				
x	0x1fc..0x1ff	4	0x1fc..0x1ff	100% <-
undeclared use (stack)				
x	0x200..0x2ff	256	-	0%

For ISA-specific state views, see the respective architecture documentation.

assert

- **Type:** String (optional)
- **Description:** Specifies the expected final state of the simulation. If the actual final state does not match, an assertion failure will be reported.
- **Example:**

```
assert: |  
    numio[0x80]: [] >>> []  
    numio[0x84]: [] >>> [120]
```

Acc32 Instruction Set Architecture (ISA) Documentation

The Acc32 ISA is a simple accumulator-based instruction set designed for educational purposes. This documentation provides an overview of the instructions available in the Acc32 ISA, their syntax, and their semantics.

Architecture Overview

The Acc32 architecture is a 32-bit accumulator-based architecture. It features:

- A single general-purpose accumulator register (Acc)
- Two status flags: overflow (V) and carry (C)
- Memory-mapped I/O
- Simple instruction set focused on accumulator operations

This architecture is excellent for beginners to understand the basics of computer architecture without the complexity of multiple registers.

Comments in Acc32 assembly code are denoted by the `;` character.

ISA Specific State Views

- `Acc:dec` , `Acc:hex` -- Acc register.
- `V` -- Overflow flag.
- `C` -- Carry flag.

Instructions

Instruction size: 1 byte for opcode, 4 bytes for absolute operand, 2 bytes for relative operand. Control flow, Load/Store Immediate/Indirect/Addr use absolute address, other -- relative.

Data Movement Instructions

- **Load Immediate**

- **Syntax:** `load_imm <value>`
- **Description:** Load an immediate value into the accumulator.
- **Operation:** `acc <- <value>`
- **Load**
 - **Syntax:** `load <offset>`
 - **Description:** Load a value from a relative address into the accumulator.
 - **Operation:** `acc <- mem[pc + <offset>]`
- **Store**
 - **Syntax:** `store <offset>`
 - **Description:** Store the accumulator value into a relative address.
 - **Operation:** `mem[pc + <offset>] <- acc`
- **Load Address**
 - **Syntax:** `load_addr <address>`
 - **Description:** Load a value from a specific address into the accumulator.
 - **Operation:** `acc <- mem[<address>]`
- **Store Address**
 - **Syntax:** `store_addr <address>`
 - **Description:** Store the accumulator value into a specific address.
 - **Operation:** `mem[<address>] <- acc`
- **Load Acc**
 - **Syntax:** `load_acc`
 - **Description:** Load a value from an address in acc into the accumulator.
 - **Operation:** `acc <- mem[acc]`
- **Store Indirect**
 - **Syntax:** `store_ind <address>`
 - **Description:** Store the accumulator value into an indirect address.
 - **Operation:** `mem[mem[<address>]] <- acc`

Arithmetic Instructions

- **Add**
 - **Syntax:** `add <address>`
 - **Description:** Add a value from a specific address to the accumulator.
 - **Operation:** `acc <- acc + mem[<address>]` and set C and V flags.
- **Subtract**
 - **Syntax:** `sub <address>`
 - **Description:** Subtract a value from a specific address from the accumulator.
 - **Operation:** `acc <- acc - mem[<address>]` and set C and V flags.
- **Multiply**
 - **Syntax:** `mul <address>`

- **Description:** Multiply the accumulator by a value from a specific address.
- **Operation:** `acc <- acc * mem[<address>]` and set C and V flags.
- **Divide**
 - **Syntax:** `div <address>`
 - **Description:** Divide the accumulator by a value from a specific address.
 - **Operation:** `acc <- acc / mem[<address>]`
- **Remainder**
 - **Syntax:** `rem <address>`
 - **Description:** Compute the remainder of the accumulator divided by a value from a specific address.
 - **Operation:** `acc <- acc % mem[<address>]`
- **Clear Overflow**
 - **Syntax:** `clv`
 - **Description:** Clear the overflow flag.
 - **Operation:** `overflow <- 0`
- **Clear Carry**
 - **Syntax:** `clc`
 - **Description:** Clear the carry flag.
 - **Operation:** `carry <- 0`

Bitwise Instructions

- **Shift Left**
 - **Syntax:** `shifl <address>`
 - **Description:** Shift the accumulator left by a number of bits from a specific address (Carry flag will not be used).
 - **Operation:** `acc <- acc << mem[<address>]`
- **Shift Right**
 - **Syntax:** `shiftr <address>`
 - **Description:** Shift the accumulator right by a number of bits from a specific address, preserving the sign (Carry flag will not be used).
 - **Operation:** `acc <- acc >> mem[<address>]`
- **Bitwise AND**
 - **Syntax:** `and <address>`
 - **Description:** Perform a bitwise AND on the accumulator with a value from a specific address.
 - **Operation:** `acc <- acc & mem[<address>]`
- **Bitwise OR**
 - **Syntax:** `or <address>`
 - **Description:** Perform a bitwise OR on the accumulator with a value from a specific address.

- **Operation:** `acc <- acc | mem[<address>]`
- **Bitwise XOR**
 - **Syntax:** `xor <address>`
 - **Description:** Perform a bitwise XOR on the accumulator with a value from a specific address.
 - **Operation:** `acc <- acc ^ mem[<address>]`
- **Bitwise NOT**
 - **Syntax:** `not`
 - **Description:** Perform a bitwise NOT on the accumulator.
 - **Operation:** `acc <- ~acc`

Control Flow Instructions

- **Jump**
 - **Syntax:** `jmp <address>`
 - **Description:** Jump to a specific address.
 - **Operation:** `pc <- <address>`
- **Branch if Equal to Zero**
 - **Syntax:** `beqz <address>`
 - **Description:** Jump to a specific address if the accumulator is zero.
 - **Operation:** `if acc == 0 then pc <- <address>`
- **Branch if Not Equal to Zero**
 - **Syntax:** `bnez <address>`
 - **Description:** Jump to a specific address if the accumulator is not zero.
 - **Operation:** `if acc != 0 then pc <- <address>`
- **Branch if Greater Than Zero**
 - **Syntax:** `bgtz <address>`
 - **Description:** Jump to a specific address if the accumulator is greater than zero.
 - **Operation:** `if acc > 0 then pc <- <address>`
- **Branch if Greater Than or Equal to Zero**
 - **Syntax:** `bgez <address>`
 - **Description:** Jump to a specific address if the accumulator is greater than or equal to zero.
 - **Operation:** `if acc >= 0 then pc <- <address>`
- **Branch if Less Than Zero**
 - **Syntax:** `bltz <address>`
 - **Description:** Jump to a specific address if the accumulator is less than zero.
 - **Operation:** `if acc < 0 then pc <- <address>`
- **Branch if Overflow Set**
 - **Syntax:** `bvs <address>`

- **Description:** Jump to a specific address if the overflow flag is set.
- **Operation:** `if overflow == 1 then pc <- <address>`
- **Branch if Overflow Clear**
 - **Syntax:** `bvc <address>`
 - **Description:** Jump to a specific address if the overflow flag is clear.
 - **Operation:** `if overflow == 0 then pc <- <address>`
- **Branch if Carry Set**
 - **Syntax:** `bcs <address>`
 - **Description:** Jump to a specific address if the carry flag is set.
 - **Operation:** `if carry == 1 then pc <- <address>`
- **Branch if Carry Clear**
 - **Syntax:** `bcc <address>`
 - **Description:** Jump to a specific address if the carry flag is clear.
 - **Operation:** `if carry == 0 then pc <- <address>`
- **Halt**
 - **Syntax:** `halt`
 - **Description:** Halt the machine.

F32a Instruction Set Architecture (ISA) Documentation

The F32a ISA is a stack-based instruction set designed for educational purposes. This documentation provides an overview of the instructions available in the F32a ISA, their syntax, and their semantics.

Architecture Overview

The F32a architecture is a 32-bit stack-based architecture inspired by Forth systems. It features:

- Two primary stacks: data stack and return stack
- Two general-purpose registers: A and B
- Stack-oriented operations that simplify many programming tasks
- Extended Arithmetic Mode (EAM) for carry-dependent operations
- Carry flag for arithmetic operations
- Memory-mapped I/O

This stack-based architecture offers a different programming paradigm compared to register-based architectures, making it excellent for understanding alternative computational models.

Comments in F32a assembly code are denoted by the `\` character.

Inspired by [F18a](#)

ISA Specific State Views

- `A:dec` , `A:hex` -- A register.
- `B:dec` , `B:hex` -- B register.
- `T:dec` , `T:hex` -- Top of the data stack.
- `S:dec` , `S:hex` -- Second value on the data stack.
- `R:dec` , `R:hex` -- Top of the return stack.
- `stack:dec` , `stack:hex` -- Entire data stack.
- `rstack:dec` , `rstack:hex` -- Entire return stack.
- `EAM` -- Extended Arithmetic Mode (1 if enabled, 0 if disabled).
- `C` -- Carry Flag (1 if set, 0 if cleared).

Stack utilization

The simulator tracks the high-water mark (deepest level reached) of each stack over the whole run and exposes it through two summary view variables. They are run-totals, so use them with `slice: last`.

- `f32a:data-stack-max` -- maximum data stack depth reached during execution.
- `f32a:return-stack-max` -- maximum return stack depth reached during execution.

Example -- print a stack-depth summary at the end of the simulation:

```
reports:
- name: stacks
  slice: last
  view: |
    f32a:data-stack-max: {f32a:data-stack-max}
    f32a:return-stack-max: {f32a:return-stack-max}
```

For `factorial.s` computing `4!` the summary reads:

```
f32a:data-stack-max: 5
f32a:return-stack-max: 3
```

The return-stack depth reflects how deeply the recursive/nested calls nested; the data-stack depth bounds how many operands were live at once -- useful for sizing the stack region and spotting runaway recursion.

Both lines are also emitted together by the generic `{isa-specific}` summary block, which lets a single report template stay uniform across ISAs.

Instructions

Instruction size: 1 byte for opcode, 4 bytes for each argument.

Extended Arithmetic Mode (EAM)

Extended Arithmetic Mode (EAM) is a mode that allows us to add with carry bit. See details in `add` instruction.

Carry Flag

Most of the instructions drop (clear) the carry flag. Specifically, any instruction that pushes to the data stack clears the carry flag as a side effect. Exceptions: `add` (sets carry flag based on overflow), `dup` (preserves carry flag).

Data Movement Instructions

- **Literal**
 - **Syntax:** `<value>` or `lit <value>`
 - **Description:** Push an immediate value onto the data stack. The `lit` keyword is optional — a bare number, hex value (`0xFF`), or character literal (`'A'`) is sufficient.
 - **Operation:** `dataStack.push(<value>)`
- **Fetch from Address**
 - **Syntax:** `@p <address>`
 - **Description:** Push the value from a specific address onto the data stack.
 - **Operation:** `dataStack.push(mem[<address>])`
- **Fetch**
 - **Syntax:** `@`
 - **Description:** Push the value from the address in register A onto the data stack.
 - **Operation:** `dataStack.push(mem[A])`
- **Fetch Plus**
 - **Syntax:** `@+`
 - **Description:** Push the value from the address in register A onto the data stack and increment A.
 - **Operation:** `dataStack.push(mem[A]); A <- A + 1`
- **Fetch B**
 - **Syntax:** `@b`
 - **Description:** Push the value from the address in register B onto the data stack.
 - **Operation:** `dataStack.push(mem[B])`
- **Store to Address**
 - **Syntax:** `!p <address>`
 - **Description:** Store the top value of the data stack into a specific address.
 - **Operation:** `mem[<address>] <- dataStack.pop()`
- **Store**
 - **Syntax:** `!`
 - **Description:** Store the top value of the data stack into the address in register A.
 - **Operation:** `mem[A] <- dataStack.pop()`
- **Store Plus**

- **Syntax:** `!+`
- **Description:** Store the top value of the data stack into the address in register A and increment A.
- **Operation:** `mem[A] <- dataStack.pop(); A <- A + 1`
- **Store B**
 - **Syntax:** `!b`
 - **Description:** Store the top value of the data stack into the address in register B.
 - **Operation:** `mem[B] <- dataStack.pop()`
- **A-Store**
 - **Syntax:** `a!`
 - **Description:** Store the top value of the data stack into register A.
 - **Operation:** `A <- dataStack.pop()`
- **B-Store**
 - **Syntax:** `b!`
 - **Description:** Store the top value of the data stack into register B.
 - **Operation:** `B <- dataStack.pop()`
- **A-Fetch**
 - **Syntax:** `a`
 - **Description:** Push the value of register A onto the data stack.
 - **Operation:** `dataStack.push(A)`

Arithmetic Instructions

- **Add**
 - **Syntax:** `+`
 - **Description:** Add the top two values on the data stack.
 - **Operation (no EAM):** `dataStack.push(dataStack.pop() + dataStack.pop()), set carry flag`
 - **Operation (EAM):** `dataStack.push(dataStack.pop() + dataStack.pop() + carry), set carry flag`
- **Multiply Step**
 - **Syntax:** `+`
 - **Description:** Perform a multiplication step.
 - **Operation:** `T <- T + (if A[0] then S else 0); A <- A >> 1; if T[0] then A[31] <- 1 else A[31] <- 0; T <- T >> 1`
- **Divide Step**
 - **Syntax:** `+/`
 - **Description:** Perform one **restoring-division** micro-step.
 - Dividend -- A
 - Divisor -- `mem[B]`
 - Partial remainder -- S

- Partial quotient -- T
- **Operation:**
 1. `S <- S << 1`
 2. `if A[31] then S[0] <- 1`
 3. `A <- A << 1`
 4. `T <- T << 1`
 5. `if S >= mem[B] then`
 - `S <- S - mem[B]`
 - `T[0] <- 1`
- [Python example](#)
- **Left Shift**
 - **Syntax:** `2*`
 - **Description:** Shift the top value of the data stack left by one bit.
 - **Operation:** `dataStack.push(dataStack.pop() << 1)`
- **Right Shift**
 - **Syntax:** `2/`
 - **Description:** Shift the top value of the data stack right by one bit, preserving the sign (Arithmetic shift).
 - **Operation:** `dataStack.push(dataStack.pop() >> 1)`
- **Invert**
 - **Syntax:** `inv`
 - **Description:** Invert all bits of the top value on the data stack.
 - **Operation:** `dataStack.push(~dataStack.pop())`
- **Extend Arithmetic Instructions**
 - **Syntax:** `eam`
 - **Description:** Pop value from the data stack and set EAM flag.
 - **Operation:** `eam <- dataStack.pop()`

Bitwise Instructions

- **And**
 - **Syntax:** `and`
 - **Description:** Perform a bitwise AND on the top two values of the data stack.
 - **Operation:** `dataStack.push(dataStack.pop() & dataStack.pop())`
- **Xor**
 - **Syntax:** `xor`
 - **Description:** Perform a bitwise XOR on the top two values of the data stack.
 - **Operation:** `dataStack.push(dataStack.pop() ^ dataStack.pop())`

Stack Manipulation Instructions

- **Drop**
 - **Syntax:** `drop`
 - **Description:** Remove the top value from the data stack.
 - **Operation:** `dataStack.pop()`
- **Dup**
 - **Syntax:** `dup`
 - **Description:** Duplicate the top value of the data stack.
 - **Operation:** `dataStack.push(dataStack.top()), carry flag does not change`
- **Over**
 - **Syntax:** `over`
 - **Description:** Copy the second value on the data stack to the top, pushing the data stack.
 - **Operation:** `T <- dataStack.pop(); S <- dataStack.pop(); dataStack.push(S); dataStack.push(T); dataStack.push(S)`

Control Flow Instructions

- **Return**
 - **Syntax:** `;`
 - **Description:** Return from a subroutine.
 - **Operation:** `p <- returnStack.pop()`
- **Call**
 - **Syntax:** `<label>`
 - **Description:** Call a subroutine at the specified label.
 - **Operation:** `returnStack.push(p); p <- <label>`
- **Jump**
 - **Syntax:** `<label> ;`
 - **Description:** Jump to the specified label.
 - **Operation:** `p <- <label>`
- **Next**
 - **Syntax:** `next <label>`
 - **Description:** Loop to the specified label, decrementing R.
 - **Operation:**
 - `if R != 0 then`
 - `R <- R - 1`
 - `p <- <label>`
 - `else returnStack.pop()`
- **If**
 - **Syntax:** `if <label>`
 - **Description:** Jump to the specified label if the top value of the data stack is zero.

- **Operation:** `if dataStack.pop() == 0 then p <- <label>`
- **Minus If**
 - **Syntax:** `-if <label>`
 - **Description:** Jump to the specified label if the top value of the data stack is nonnegative.
 - **Operation:** `if dataStack.pop() >= 0 then p <- <label>`
- **Halt**
 - **Syntax:** `halt`
 - **Description:** Halt the machine.

Register Manipulation Instructions

- **RIntoT**
 - **Syntax:** `r>`
 - **Description:** Move the top value of the return stack to the data stack.
 - **Operation:** `dataStack.push(returnStack.pop())`
- **TIntoR**
 - **Syntax:** `>r`
 - **Description:** Move the top value of the data stack to the return stack.
 - **Operation:** `returnStack.push(dataStack.pop())`

M68k Instruction Set Architecture (ISA) Documentation

The M68k ISA is a simple CISC-like instruction set inspired by the Motorola 68000 architecture. This documentation provides an overview of the instructions available in the M68k ISA, their syntax, and their semantics.

Architecture Overview

The M68k architecture is a 32-bit CISC (Complex Instruction Set Computer) architecture inspired by the Motorola 68000 series. It features:

- 8 general-purpose data registers (D0-D7)
- 8 address registers (A0-A7, with A7 serving as the stack pointer)
- A variety of addressing modes for flexible memory access
- Rich set of instructions with multiple operand sizes
- Status register with condition code flags
- Memory-mapped I/O

This architecture provides a balance between the simplicity of RISC designs and the feature-rich nature of CISC processors, making it excellent for understanding more complex processor designs.

Comments in M68k assembly code are denoted by the `;` character.

- [68000 INSTRUCTION SET](#)
- [Wiki: Motorola 68000](#)

ISA Specific State Views

- `D<n>:dec` , `D<n>:hex` -- Data registers (D0-D7).
- `A<n>:dec` , `A<n>:hex` -- Address registers (A0-A7).
- `PC:dec` , `PC:hex` -- Program Counter.
- `SR:bin` -- Status Register (flags: N, Z, V, C).

Memory and Register Usage

- **Data Registers (D0-D7):** General-purpose registers for arithmetic, logical, and data manipulation.
- **Address Registers (A0-A7):** Used for addressing memory. A7 is also the stack pointer (SP).
- **Status Register (SR):** Contains condition codes (flags):
 - **N (Negative):** Set if the result of an operation is negative.
 - **Z (Zero):** Set if the result of an operation is zero.
 - **V (Overflow):** Set if an arithmetic overflow occurs.
 - **C (Carry):** Set if a carry or borrow occurs in arithmetic operations.

Instructions

Instruction sizes vary depending on the operation mode and addressing modes used. The `ByteSize` calculation now takes into account differences between byte and long operations, with byte operations generally using less memory than long operations.

Operation Modes

M68k instructions can operate on data of different sizes:

- **Byte (`.b`):** 8-bit operations
- **Long (`.l`):** 32-bit operations

The size suffix is mandatory and must be explicitly specified for all instructions that support it.

Example: `add.b D0, D1` operates on the least significant byte of D0 and D1.

Note on ByteSize implementation: The `ByteSize` implementation for instructions now differentiates between byte and long operations. Byte operations (`.b`) use 1 byte for data, while long operations (`.l`) use 4 bytes. All jump and branch instructions always use long (6-byte) encodings regardless of the displacement.

Addressing Modes

The M68k ISA supports the following addressing modes:

1. Register Direct

- **Data Register Direct:** D0 through D7 -- usable as source or destination for every instruction that takes an operand.
- **Address Register Direct:** A0 through A7 -- usable only as movea's source/destination and as the address-register operand of link/unlk. Other instructions reject An in the operand slot with a parser error; reach an address register's value via movea first, or use one of the indirect modes below.

2. Immediate: value

- Example: `move.l 42, D0`

3. Memory Addressing

- **Address Register Indirect:** (An)
 - Accesses memory at the address contained in An
 - Example: `move.l (A0), D0`
- **Address Register Indirect with Postincrement:** (An)+
 - Uses the address in An, then increments An by the size of operation (1 for byte, 4 for long)
 - Example: `move.l (A0)+, D0`
- **Address Register Indirect with Predecrement:** -(An)
 - Decrements An by the size of operation, then uses the resulting address
 - Example: `move.l -(A0), D0`
- **Address Register Indirect with Displacement:** d(An)
 - Uses the address in An plus a signed 16-bit displacement d
 - Example: `move.l 8(A0), D0` or `move.l -4(A0), D0`
- **Address Register Indirect with Index:** d(An,Xn)
 - Uses the address in An plus the value in Xn (which can be a data or address register) plus a signed displacement d
 - Example: `move.l 4(A0,D1), D0` or `move.l 8(A0,A1), D0`

Data Movement Instructions

• Move

- **Syntax:** `move.<size> <source>, <destination>`
- **Description:** Move data from the source to the destination. Sets N and Z flags based on the result, clears V and C flags.
- **Operation:** `<destination> <- <source>`
- **Examples:**

```
move.l D0, D1      ; Copy D0 to D1 (32-bit)
move.b D0, D1      ; Copy lowest byte from D0 to lowest byte of
D1
move.l 42, D0       ; Load immediate value 42 into D0
```

```

move.l (A0), D0      ; Load value from memory address in A0 into D0
move.l D0, (A1)      ; Store value from D0 into memory address in
A1
move.l (A0)+, D0     ; Load value from A0, then increment A0 by 4
move.l -(A0), D0     ; Decrement A0 by 4, then load value from A0
move.l 8(A0), D0     ; Load value from A0+8
move.l -4(A0), D0    ; Load value from A0-4
move.l 8(A0,D1), D0  ; Load value from A0+D1+8
move.l 4(A0,A1), D0  ; Load value from A0+A1+4

```

- **Move to Address Register**

- **Syntax:** `movea.l <source>, <destination>`
- **Description:** Move data from the source to the destination address register. `movea` is the only instruction that accepts `An` as a direct source operand -- other instructions reject `An` at parse time. `movea` is long-only (no `.b` form). Does not affect flags.
- **Operation:** `<destination> <- <source>`
- **Examples:**

```

movea.l 1000, A0     ; Load immediate value 1000 into A0
movea.l D0, A1       ; Copy D0 to A1
movea.l A2, A0       ; Copy A2 to A0 (only place An is accepted as
source)

```

Logical Instructions

- **Clear**

- **Syntax:** `clr.<size> <destination>`
- **Description:** Clear the destination to zero. Sets Z flag, clears N, V, and C flags.
- **Operation:** `<destination> <- 0`
- **Examples:**

```

clr.l D0             ; D0 <- 0 (32-bit)
clr.b D0             ; Clear lowest byte of D0

```

- **Negate**

- **Syntax:** `neg.<size> <destination>`
- **Description:** Negate the destination (compute $0 - \text{destination}$). Sets N, Z, V, and C flags based on the result.
- **Operation:** `<destination> <- 0 - <destination>`
- **Examples:**

```

neg.l D0             ; D0 <- -D0 (32-bit)
neg.b D0             ; Negate lowest byte of D0

```

- **Not**

- **Syntax:** `not.<size> <destination>`

- **Description:** Bitwise NOT the destination. Sets N and Z flags based on the result, clears V and C flags.
- **Operation:** `<destination> <- ~<destination>`
- **Examples:**

```
not.l D0          ; D0 <- ~D0 (32-bit)
not.b D0          ; Apply NOT to lowest byte of D0
```

- **And**

- **Syntax:** `and.<size> <source>, <destination>`
- **Description:** Bitwise AND the source with the destination. Sets N and Z flags based on the result, clears V and C flags.
- **Operation:** `<destination> <- <destination> & <source>`
- **Examples:**

```
and.l D0, D1      ; D1 <- D1 & D0 (32-bit)
and.b D0, D1      ; AND lowest bytes of D0 and D1
```

- **Or**

- **Syntax:** `or.<size> <source>, <destination>`
- **Description:** Bitwise OR the source with the destination. Sets N and Z flags based on the result, clears V and C flags.
- **Operation:** `<destination> <- <destination> | <source>`
- **Examples:**

```
or.l D0, D1       ; D1 <- D1 | D0 (32-bit)
or.b D0, D1       ; OR lowest bytes of D0 and D1
```

- **Exclusive Or**

- **Syntax:** `xor.<size> <source>, <destination>`
- **Description:** Bitwise XOR the source with the destination. Sets N and Z flags based on the result, clears V and C flags.
- **Operation:** `<destination> <- <destination> ^ <source>`
- **Examples:**

```
xor.l D0, D1      ; D1 <- D1 ^ D0 (32-bit)
xor.b D0, D1      ; XOR lowest bytes of D0 and D1
```

Arithmetic Instructions

- **Add**

- **Syntax:** `add.<size> <source>, <destination>`
- **Description:** Add the source to the destination.
- **Operation:** `<destination> <- <destination> + <source>`
- **Examples:**

```
add.l D0, D1      ; D1 <- D1 + D0 (32-bit)
add.b D0, D1      ; Add lowest bytes of D0 and D1
add.l 10, D0      ; D0 <- D0 + 10
add.b 5, D0       ; Add 5 to lowest byte of D0
```

- **Subtract**

- **Syntax:** `sub.<size> <source>, <destination>`
- **Description:** Subtract the source from the destination.
- **Operation:** `<destination> <- <destination> - <source>`
- **Examples:**

```
sub.l D0, D1      ; D1 <- D1 - D0 (32-bit)
sub.b D0, D1      ; Subtract lowest bytes of D0 from D1
sub.l 1, D0       ; D0 <- D0 - 1
sub.b 1, D0       ; Subtract 1 from lowest byte of D0
```

- **Compare**

- **Syntax:** `cmp.<size> <source>, <destination>`
- **Description:** Compare the destination with the source by subtracting the source from the destination and setting the condition codes accordingly, without storing the result.
- **Operation:** `temp <- <destination> - <source>` (sets flags only)
- **Examples:**

```
cmp.l D0, D1      ; Compare D1 with D0 (32-bit)
cmp.b D0, D1      ; Compare lowest bytes of D1 and D0
cmp.l 10, D0      ; Compare D0 with immediate value 10
cmp.b 5, D0       ; Compare lowest byte of D0 with 5
cmp.l (A0), D0    ; Compare D0 with memory at address in A0
```

- **Multiply**

- **Syntax:** `mul.<size> <source>, <destination>`
- **Description:** Multiply the destination by the source.
- **Operation:** `<destination> <- <destination> * <source>`
- **Examples:**

```
mul.l D0, D1      ; D1 <- D1 * D0 (32-bit)
mul.b D0, D1      ; Multiply lowest bytes of D0 and D1
```

- **Divide**

- **Syntax:** `div.<size> <source>, <destination>`
- **Description:** Divide the destination by the source.
- **Operation:** `<destination> <- <destination> / <source>`
- **Examples:**

```
div.l D0, D1      ; D1 <- D1 / D0 (32-bit)
```

```
div.b D0, D1 ; Divide using lowest bytes of registers
```

Shift and Rotate Instructions

- **Arithmetic Shift Left**

- **Syntax:** `asl.<size> <source>, <destination>`
- **Description:** Shift the destination left arithmetically by the count specified in the source. Sets N and Z flags based on the result, C flag is set to the last bit shifted out (0 if shift count is 0), clears V flag.
- **Operation:** `<destination> <- <destination> << <source>`
- **Examples:**

```
asl.l 2, D0 ; D0 <- D0 << 2 (32-bit)
asl.b 2, D0 ; Shift lowest byte of D0 left by 2 bits
asl.l D1, D0 ; D0 <- D0 << D1
```

- **Arithmetic Shift Right**

- **Syntax:** `asr.<size> <source>, <destination>`
- **Description:** Shift the destination right arithmetically by the count specified in the source. Sets N and Z flags based on the result, C flag is set to the last bit shifted out (0 if shift count is 0), clears V flag.
- **Operation:** `<destination> <- <destination> >> <source>` (sign-preserving)
- **Examples:**

```
asr.l 2, D0 ; D0 <- D0 >> 2 (32-bit)
asr.b 2, D0 ; Shift lowest byte of D0 right by 2 bits
asr.l D1, D0 ; D0 <- D0 >> D1
```

- **Logical Shift Left**

- **Syntax:** `lsl.<size> <source>, <destination>`
- **Description:** Shift the destination left logically by the count specified in the source. Sets N and Z flags based on the result, C flag is set to the last bit shifted out (0 if shift count is 0), clears V flag.
- **Operation:** `<destination> <- <destination> << <source>`
- **Examples:**

```
lsl.l 2, D0 ; D0 <- D0 << 2 (32-bit)
lsl.b 2, D0 ; Shift lowest byte of D0 left by 2 bits
lsl.l D1, D0 ; D0 <- D0 << D1
```

- **Logical Shift Right**

- **Syntax:** `lsr.<size> <source>, <destination>`
- **Description:** Shift the destination right logically by the count specified in the source. Sets N and Z flags based on the result, C flag is set to the last bit shifted out (0 if shift count is 0), clears V flag.
- **Operation:** `<destination> <- <destination> >> <source>` (zero-fill)

- **Examples:**

```
lsr.l 2, D0      ; D0 <- D0 >> 2 (32-bit)
lsr.b 2, D0      ; Shift lowest byte of D0 right by 2 bits
lsr.l D1, D0     ; D0 <- D0 >> D1
```

Control Flow Instructions

- **Jump**

- **Syntax:** `jmp <label>`
- **Description:** Jump to the specified label.
- **Operation:** `PC <- <label>`
- **Examples:**

```
jmp loop          ; Jump to label "loop"
```

- **Branch if Carry Clear**

- **Syntax:** `bcc <label>`
- **Description:** Branch to the specified label if the carry flag is clear.
- **Operation:** `if C == 0 then PC <- <label>`
- **Examples:**

```
bcc loop          ; Jump to "loop" if C flag is clear
```

- **Branch if Carry Set**

- **Syntax:** `bcs <label>`
- **Description:** Branch to the specified label if the carry flag is set.
- **Operation:** `if C == 1 then PC <- <label>`
- **Examples:**

```
bcs error         ; Jump to "error" if C flag is set
```

- **Branch if Equal**

- **Syntax:** `beq <label>`
- **Description:** Branch to the specified label if the zero flag is set.
- **Operation:** `if Z == 1 then PC <- <label>`
- **Examples:**

```
beq done          ; Jump to "done" if Z flag is set
```

- **Branch if Not Equal**

- **Syntax:** `bne <label>`
- **Description:** Branch to the specified label if the zero flag is clear.
- **Operation:** `if Z == 0 then PC <- <label>`
- **Examples:**

```
bne loop ; Jump to "loop" if Z flag is clear
```

- **Branch if Less Than**

- **Syntax:** `blt <label>`
- **Description:** Branch to the specified label if $N \neq V$.
- **Operation:** `if  $N \neq V$  then  $PC \leftarrow \text{<label>}$`
- **Examples:**

```
blt less ; Jump to "less" if result is less than
```

- **Branch if Greater Than**

- **Syntax:** `bgt <label>`
- **Description:** Branch to the specified label if $Z = 0$ and $N = V$.
- **Operation:** `if  $Z == 0$  and  $N == V$  then  $PC \leftarrow \text{<label>}$`
- **Examples:**

```
bgt greater ; Jump to "greater" if result is greater than
```

- **Branch if Less Than or Equal**

- **Syntax:** `ble <label>`
- **Description:** Branch to the specified label if $Z = 1$ or $N \neq V$.
- **Operation:** `if  $Z == 1$  or  $N \neq V$  then  $PC \leftarrow \text{<label>}$`
- **Examples:**

```
ble less_or_equal ; Jump to "less_or_equal" if result is less  
than or equal
```

- **Branch if Greater Than or Equal**

- **Syntax:** `bge <label>`
- **Description:** Branch to the specified label if $N = V$.
- **Operation:** `if  $N == V$  then  $PC \leftarrow \text{<label>}$`
- **Examples:**

```
bge greater_or_equal ; Jump to "greater_or_equal" if result is  
greater than or equal
```

- **Branch if Minus**

- **Syntax:** `bmi <label>`
- **Description:** Branch to the specified label if the negative flag is set.
- **Operation:** `if  $N == 1$  then  $PC \leftarrow \text{<label>}$`
- **Examples:**

```
bmi negative ; Jump to "negative" if result is negative
```

- **Branch if Plus**

- **Syntax:** `bpl <label>`

- **Description:** Branch to the specified label if the negative flag is clear.

- **Operation:** if N == 0 then PC <- <label>

- **Examples:**

```
bpl positive      ; Jump to "positive" if result is positive or zero
```

- **Branch if Overflow Clear**

- **Syntax:** bvc <label>

- **Description:** Branch to the specified label if the overflow flag is clear.

- **Operation:** if V == 0 then PC <- <label>

- **Examples:**

```
bvc no_overflow   ; Jump to "no_overflow" if V flag is clear
```

- **Branch if Overflow Set**

- **Syntax:** bvs <label>

- **Description:** Branch to the specified label if the overflow flag is set.

- **Operation:** if V == 1 then PC <- <label>

- **Examples:**

```
bvs overflow      ; Jump to "overflow" if V flag is set
```

- **Jump to Subroutine**

- **Syntax:** jsr <label>

- **Description:** Jump to a subroutine by saving the return address on the stack and jumping to the target address.

- **Operation:** Push PC to stack, then PC <- target address

- **Examples:**

```
jsr my_function   ; Call subroutine at my_function
```

- **Return from Subroutine**

- **Syntax:** rts

- **Description:** Return from a subroutine by popping the return address from the stack and jumping to it.

- **Operation:** Pop PC from stack

- **Examples:**

```
rts               ; Return from subroutine
```

- **Link**

- **Syntax:** link <address_register>, <offset>

- **Description:** Creates a stack frame for a subroutine by establishing a new frame pointer and allocating stack space for local variables. The link instruction is

typically used at the beginning of a function to set up the environment for local variable storage.

- **Detailed Operation:**

1. Push the current value of the specified address register (typically A6, used as frame pointer) onto the stack
2. Copy the current stack pointer (A7) value into the specified address register, establishing a new frame pointer
3. Add the offset value to the stack pointer (A7) to reserve space for local variables
 - The offset should be negative to allocate space (e.g., -4 allocates 4 bytes)
 - Local variables can then be accessed using negative offsets from the frame pointer

- **Usage Context:**

- Commonly used with A6 as the conventional frame pointer register
- Local variables are accessed via frame pointer with negative offsets (e.g., -4(A6))
- Parameters passed on the stack can be accessed via positive offsets (e.g., 8(A6))

- **Examples:**

```
link A6, -8      ; Save old A6 on stack, set A6 = SP, subtract 8
                  from SP to allocate space
move.l D0, -4(A6) ; Store D0 in the first local variable (4 bytes
                  from frame pointer)
move.l D1, -8(A6) ; Store D1 in the second local variable (8
                  bytes from frame pointer)
```

- **Unlink**

- **Syntax:** `unlk <address_register>`
- **Description:** Deallocates a stack frame at the end of a subroutine, restoring both the stack pointer and frame pointer to their values before the corresponding `link` instruction was executed. The `unlk` instruction is typically the last operation before returning from a function.
- **Detailed Operation:**
 1. Copy the frame pointer (specified address register, typically A6) to the stack pointer (A7)
 - This effectively removes all local variables by restoring the stack pointer to its pre-allocation position
 2. Pop the value from the top of stack into the specified address register
 - This restores the previous frame pointer value that was saved by the `link` instruction
- **Usage Context:**
 - Always paired with a corresponding `link` instruction
 - Typically followed by an `rts` (return from subroutine) instruction

- Properly deallocates all local variables created in the function
- **Examples:**

```
unlk A6          ; Set SP = A6 (deallocating locals), then pop
saved frame pointer into A6
rts              ; Return from subroutine
```

Complete function example:

```
my_function:
    link A6, -12      ; Create stack frame with 12 bytes for
local variables
    move.l 8(A6), D0   ; Access parameter passed on stack (at
positive offset)
    move.l D0, -4(A6)  ; Store in local variable (at negative
offset)
    ; ... function body ...
    unlk A6           ; Destroy stack frame
    rts               ; Return from subroutine
```

- **Halt**
 - **Syntax:** `halt`
 - **Description:** Halt the machine.
 - **Operation:** Stop execution
 - **Examples:**

```
halt              ; Stop the program
```

RISC-IV Instruction Set Architecture (ISA) Documentation

The RISC-IV ISA is a simple register-based instruction set inspired by the RISC-V architecture. This documentation provides an overview of the instructions available in the RISC-IV ISA, their syntax, and their semantics.

Architecture Overview

The RISC-IV architecture is a 32-bit RISC (Reduced Instruction Set Computer) architecture inspired by the RISC-V specification. It features:

- 32 general-purpose registers (including one hardwired zero register — writes to Zero are silently discarded)
- Fixed-length 4-byte instructions
- Load-store architecture (memory access only through specific instructions)
- Simple addressing modes
- Memory-mapped I/O

- Support for function calls and returns through jump-and-link instructions
- Arithmetic, logical, and control flow operations

This architecture provides a clean, orthogonal instruction set that exemplifies RISC design principles, making it excellent for educational purposes while still being powerful enough for practical applications.

Comments in RISC-IV assembly code are denoted by the `;` character.

Inspired by [RISC-V](#)

Register Usage Conventions

Although most registers can technically be used for any purposes, the following conventions are recommended

Register(s)	Purpose	Convention
Zero	Constant zero value. Writes are ignored	Preserved
Ra	Return address for function calls	Caller-saved
Sp	Stack pointer	Callee-saved
Gp	Global pointer. Points to a region containing frequently accessed global data	Callee-saved
Tp	Thread pointer. Reserved for thread-local data	Callee-saved
A0 - A7	Function arguments and return values	Caller-saved
T0 - T6	Temporary registers for intermediate calculations	Caller-saved
S0Fp	Frame pointer or saved register	Callee-saved
S1 - S11	Saved registers for long-lived values	Callee-saved

Caller-saved registers may be freely modified by the called function. If the caller needs their values after a function call, it must save and restore them

Callee-saved registers must retain their values across function calls. A function that modifies a callee-saved register must restore its original value before returning

Immediate Value Relocation Directives

The RISC-IV assembly language provides special directives for handling larger immediate values that don't fit within the standard instruction formats:

- `%hi(symbol)`

- **Description:** Used to extract the upper 20 bits of a 32-bit address or immediate value
- **Usage:** `lui rd, %hi(symbol)`
- **Operation:** `rd <- (symbol & 0xFFFFF000)`
- **%lo(symbol)**
 - **Description:** Used to extract the lower 12 bits of a 32-bit address or immediate value
 - **Usage:** `addi rd, rs, %lo(symbol)`
 - **Operation:** `rd <- rs + (symbol & 0x00000FFF)`

These directives are typically used together to load a full 32-bit address into a register:

```
lui a0, %hi(address) ; Load upper 20 bits into a0
addi a0, a0, %lo(address) ; Add lower 12 bits to a0
```

Instructions

Instruction size: 4 bytes.

Data Movement Instructions

- **Load Upper Immediate**
 - **Syntax:** `lui <rd>, <k>`
 - **Description:** Load an immediate value masked to 20 bits and shifted left by 12 bits into the destination register.
 - **Operation:** `rd <- (k & 0x000FFFFF) << 12`
- **Move**
 - **Syntax:** `mv <rd>, <rs>`
 - **Description:** Move the value from the source register to the destination register.
 - **Operation:** `rd <- rs`
- **Store Word**
 - **Syntax:** `sw <rs2>, <offset>(<rs1>)`
 - **Description:** Store the value from the source register into memory at the address computed by adding the offset to the base register.
 - **Operation:** `M[offset + rs1] <- rs2`
- **Store Byte**
 - **Syntax:** `sb <rs2>, <offset>(<rs1>)`
 - **Description:** Store the lower 8 bits of the value from the source register into memory at the address computed by adding the offset to the base register.
 - **Operation:** `M[offset + rs1] <- rs2 & 0xFF`
- **Load Word**
 - **Syntax:** `lw <rd>, <offset>(<rs1>)`

- **Description:** Load a word from memory at the address computed by adding the offset to the base register into the destination register.
- **Operation:** `rd <- M[offset + rs1]`
- **Load Byte**
 - **Syntax:** `lb <rd>, <offset>(<rs1>)`
 - **Description:** Load a byte from memory at the address computed by adding the offset to the base register, sign-extend it to 32 bits, and store in the destination register.
 - **Operation:** `rd <- signext(M[offset + rs1][7:0])`

Arithmetic Instructions

- **Add Immediate**
 - **Syntax:** `addi <rd>, <rs1>, <k>`
 - **Description:** Add a 12-bit sign-extended immediate value to the source register and store the result in the destination register. The immediate `k` is truncated to 12 bits and sign-extended to 32 bits before the addition.
 - **Operation:** `rd <- rs1 + signext(k[11:0])`
- **Set Less Than Immediate**
 - **Syntax:** `slti <rd>, <rs1>, <k>`
 - **Description:** Set the destination register to 1 if the source register is less than the immediate value (signed comparison), else set to 0.
 - **Operation:** `rd <- (rs1 < k) ? 1 : 0`
- **Add**
 - **Syntax:** `add <rd>, <rs1>, <rs2>`
 - **Description:** Add the values of two source registers and store the result in the destination register.
 - **Operation:** `rd <- rs1 + rs2`
- **Subtract**
 - **Syntax:** `sub <rd>, <rs1>, <rs2>`
 - **Description:** Subtract the value of the second source register from the first source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 - rs2`
- **Multiply**
 - **Syntax:** `mul <rd>, <rs1>, <rs2>`
 - **Description:** Multiply the values of two source registers and store the result in the destination register.
 - **Operation:** `rd <- rs1 * rs2`
- **Multiply High**
 - **Syntax:** `mulh <rd>, <rs1>, <rs2>`

- **Description:** Multiply the values of two source registers and store the high part of the result in the destination register.
- **Operation:** `rd <- (rs1 * rs2) >> (word size)`
- **Divide**
 - **Syntax:** `div <rd>, <rs1>, <rs2>`
 - **Description:** Divide the value of the first source register by the value of the second source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 / rs2`
- **Remainder**
 - **Syntax:** `rem <rd>, <rs1>, <rs2>`
 - **Description:** Compute the remainder of the division of the first source register by the second source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 % rs2`

Bitwise Instructions

- **Logical Shift Left Immediate**
 - **Syntax:** `slli <rd>, <rs1>, <k>`
 - **Description:** Shift the value of the source register left by the immediate amount (lower 5 bits) and store the result in the destination register.
 - **Operation:** `rd <- rs1 << (k & 0x1F)`
- **Logical Shift Right Immediate**
 - **Syntax:** `srl <rd>, <rs1>, <k>`
 - **Description:** Shift the value of the source register right (zero-fill) by the immediate amount (lower 5 bits) and store the result in the destination register.
 - **Operation:** `rd <- rs1 >>> (k & 0x1F)`
- **Arithmetic Shift Right Immediate**
 - **Syntax:** `srai <rd>, <rs1>, <k>`
 - **Description:** Shift the value of the source register right by the immediate amount (lower 5 bits), preserving the sign, and store the result in the destination register.
 - **Operation:** `rd <- rs1 >> (k & 0x1F)`
- **Logical Shift Left**
 - **Syntax:** `sll <rd>, <rs1>, <rs2>`
 - **Description:** Shift the value of the first source register left by the number of bits specified in the lower 5 bits of the second source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 << (rs2 & 0x1F)`
- **Logical Shift Right**
 - **Syntax:** `srl <rd>, <rs1>, <rs2>`
 - **Description:** Shift the value of the first source register right by the number of bits specified in the lower 5 bits of the second source register and store the result in the

destination register.

- **Operation:** `rd <- rs1 >> (rs2 & 0x1F)`

- **Arithmetic Shift Right**

- **Syntax:** `sra <rd>, <rs1>, <rs2>`
- **Description:** Shift the value of the first source register right by the number of bits specified in the lower 5 bits of the second source register, preserving the sign, and store the result in the destination register.
- **Operation:** `rd <- rs1 >> (rs2 & 0x1F)`

- **Bitwise AND**

- **Syntax:** `and <rd>, <rs1>, <rs2>`
- **Description:** Perform a bitwise AND on the values of two source registers and store the result in the destination register.
- **Operation:** `rd <- rs1 & rs2`

- **Bitwise AND Immediate**

- **Syntax:** `andi <rd>, <rs1>, <k>`
- **Description:** Perform a bitwise AND of the source register with a 12-bit sign-extended immediate value.
- **Operation:** `rd <- rs1 & signext(k[11:0])`

- **Bitwise OR**

- **Syntax:** `or <rd>, <rs1>, <rs2>`
- **Description:** Perform a bitwise OR on the values of two source registers and store the result in the destination register.
- **Operation:** `rd <- rs1 | rs2`

- **Bitwise OR Immediate**

- **Syntax:** `ori <rd>, <rs1>, <k>`
- **Description:** Perform a bitwise OR of the source register with a 12-bit sign-extended immediate value.
- **Operation:** `rd <- rs1 | signext(k[11:0])`

- **Bitwise XOR**

- **Syntax:** `xor <rd>, <rs1>, <rs2>`
- **Description:** Perform a bitwise XOR on the values of two source registers and store the result in the destination register.
- **Operation:** `rd <- rs1 ^ rs2`

- **Bitwise XOR Immediate**

- **Syntax:** `xori <rd>, <rs1>, <k>`
- **Description:** Perform a bitwise XOR of the source register with a 12-bit sign-extended immediate value.
- **Operation:** `rd <- rs1 ^ signext(k[11:0])`

Control Flow Instructions

- **Jump**
 - **Syntax:** `j <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter.
 - **Operation:** `pc <- pc + k`
- **Jump and Link**
 - **Syntax:** `jal <rd>, <k>`
 - **Description:** Store the address of the next instruction in the destination register and jump to the address computed by adding the immediate value to the current program counter.
 - **Operation:** `rd <- pc + 4, pc <- pc + k`
- **Jump Register**
 - **Syntax:** `jr <rs>`
 - **Description:** Jump to the address stored in the source register.
 - **Operation:** `pc <- rs`
- **Branch if Equal to Zero**
 - **Syntax:** `beqz <rs1>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the source register is zero.
 - **Operation:** `if rs1 == 0 then pc <- pc + k`
- **Branch if Not Equal to Zero**
 - **Syntax:** `bnez <rs1>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the source register is not zero.
 - **Operation:** `if rs1 != 0 then pc <- pc + k`
- **Branch if Greater Than**
 - **Syntax:** `bgt <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is greater than the value in the second source register.
 - **Operation:** `if rs1 > rs2 then pc <- pc + k`
- **Branch if Less Than or Equal**
 - **Syntax:** `ble <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is less than or equal to the value in the second source register.
 - **Operation:** `if rs1 <= rs2 then pc <- pc + k`
- **Branch if Greater Than (Unsigned)**
 - **Syntax:** `bgtu <rs1>, <rs2>, <k>`

- **Description:** Jump to the address computed by adding the immediate value to the current program counter if the unsigned interpretation of the first source register is greater than the unsigned interpretation of the second source register.
- **Operation:** `if unsigned(rs1) > unsigned(rs2) then pc <- pc + k`
- **Branch if Less Than or Equal (Unsigned)**
 - **Syntax:** `bleu <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the unsigned interpretation of the first source register is less than or equal to the unsigned interpretation of the second source register.
 - **Operation:** `if unsigned(rs1) <= unsigned(rs2) then pc <- pc + k`
- **Branch if Equal**
 - **Syntax:** `beq <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is equal to the value in the second source register.
 - **Operation:** `if rs1 == rs2 then pc <- pc + k`
- **Branch if Not Equal**
 - **Syntax:** `bne <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is not equal to the value in the second source register.
 - **Operation:** `if rs1 != rs2 then pc <- pc + k`
- **Halt**
 - **Syntax:** `halt`
 - **Description:** Halt the machine.

ISA Specific State Views

- `<reg>:dec , <reg>:hex` -- View the value of a specific register in decimal or hexadecimal format.

Available registers: Zero , Ra , Sp , Gp , Tp , T0 , T1 , T2 , S0Fp , S1 , A0 , A1 , A2 , A3 , A4 , A5 , A6 , A7 , S2 , S3 , S4 , S5 , S6 , S7 , S8 , S9 , S10 , S11 , T3 , T4 , T5 , T6 .

VLIW-IV Instruction Set Architecture (ISA) Documentation

The VLIW ISA is a simple register-based instruction set inspired by VLIW (Very Long Instruction Word) principles and RISC-V. This documentation provides an overview of the instructions available in the VLIW ISA, their syntax, and their semantics.

Architecture Overview

The VLIW architecture is a 32-bit VLIW (Very Long Instruction Word) architecture inspired by RISC-V and classic VLIW designs. It features:

- 32 general-purpose registers (including one hardwired zero register — writes to Zero are silently ignored)
- Fixed-length 14-byte (112-bit) instruction bundles, divided into 4 slots for parallel execution
- Load-store architecture (memory access only through specific instructions in dedicated slots)
- Simple addressing modes
- Memory-mapped I/O
- Support for function calls and returns through jump-and-link instructions
- Arithmetic, logical, and control flow operations executed in parallel where possible
- Static scheduling: The compiler/assembler bundles independent operations; hardware executes them in lockstep

This architecture emphasizes instruction-level parallelism (ILP) through wide instructions, making it ideal for educational exploration of VLIW concepts like compiler scheduling, slot utilization, and parallelism trade-offs, while maintaining RISC-like simplicity in individual operations.

Comments in VLIW assembly code are denoted by the `;` character.

Inspired by [RISC-V](#) and classic VLIW designs.

Immediate Value Relocation Directives

The VLIW assembly language provides special directives for handling larger immediate values that don't fit within the standard instruction formats similar to RISC-IV implementation:

- **%hi(symbol)**
 - **Description:** Used to extract the upper 20 bits of a 32-bit address or immediate value
 - **Usage:** `lui rd, %hi(symbol) / nop / nop / nop`
 - **Operation:** `rd <- (symbol & 0xFFFFF000)`
- **%lo(symbol)**
 - **Description:** Used to extract the lower 12 bits of a 32-bit address or immediate value
 - **Usage:** `addi rd, rs, %lo(symbol) / nop / nop / nop`
 - **Operation:** `rd <- rs + (symbol & 0x00000FFF)`

These directives are typically used together to load a full 32-bit address into a register:

```
lui a0, %hi(address) / nop / nop / nop ; Load upper 20 bits into a0
```

```
addi a0, a0, %lo(address) / nop / nop / nop ; Add lower 12 bits to a0
```

Instructions

Instruction size: 14 bytes (112-bit bundle).

```
Bundle: [ALU1:32] [ALU2:32] [MEM:24] [CTRL:24]

[ALU1/ALU2]:
  [opcode:5] [rd:5] [rs1:5] [rs2:5] [reserved:12] ; add sub mul mulh div
rem sll srl sra and or xor
  [opcode:5] [rd:5] [rs1:5] [imm_sign:17] ; addi slti
  [opcode:5] [rd:5] [imm_unsign:22] ; lui
  [opcode:5] [rd:5] [rs1:5] [reserved:17] ; mv
  [opcode:5] [reserved:27] ; nop
[MEM] (24 bits):
  [opcode:3] [rd:5] [base:5] [offset_sign:11] ; lw lb (reg =
rd)
  [opcode:3] [rs2:5] [base:5] [offset_sign:11] ; sw sb (reg =
rs2)
  [opcode:3] [reserved:21] ; nop
[CTRL] (24 bits):
  [opcode:4] [rs1:5] [rs2:5] [offset_sign:10] ; beq bne blt bgt ble
bgtu bleu
  [opcode:4] [rs1:5] [offset_sign:15] ; beqz bnez
  [opcode:4] [rd:5] [offset_sign:15] ; jal
  [opcode:4] [rs:5] [reserved:15] ; jr
  [opcode:4] [offset_sign:20] ; j
  [opcode:4] [reserved:20] ; nop halt
```

The two ALU slots are 32 bits each; MEM and CTRL are 24 bits each, for a 112-bit (14-byte) bundle. The opcode occupies the high bits of its slot, sized to the number of operations the slot

defines: 5 bits for ALU (17 ops), 4 for CTRL (14 ops), 3 for MEM (5 ops). Register fields follow the opcode; the remaining low bits hold the immediate/offset.

Every immediate and offset is **silently truncated** to its field width: the value is reduced to the low bits of the field and sign-extended (lui's imm_unsign is zero-extended), with no error if it does not fit. So addi / slti see a 17-bit signed immediate, branches a 10-bit signed offset, beqz / bnez / jal a 15-bit one, j a 20-bit one, and memory ops an 11-bit one. lui keeps only the low 20 bits (after the << 12 shift the upper bits of the 22-bit field cannot be represented in a 32-bit word). The %lo (12-bit) and %hi (20-bit) relocation values fit within these fields unchanged.

Each instruction is a bundle with 4 slots: Slot 0 (ALU1), Slot 1 (ALU2), Slot 2 (Memory), Slot 3 (Control). Operations in slots execute in parallel. Unused slots are NOP (no operation). Assembly syntax uses / to separate slots.

Execution model: All source operands are read first, then all results are written back simultaneously. The order of register writes from ALU1, ALU2, and Memory slots is non-deterministic — if multiple slots write to the same register, the result is undefined. The control slot always executes last. The compiler/assembler must ensure that parallel operations in the same bundle are independent (no write-write or read-write conflicts across slots).

```
add rd, rs1, rs2 / addi rd, rs1, k / lw rd, offset(rs1) / beq rs1, rs2, k
```

Slot 0 and Slot 1: ALU Operations (Identical)

- **Load Upper Immediate**
 - **Syntax:** `lui <rd>, <k>`
 - **Description:** Load an immediate value masked to 20 bits and shifted left by 12 bits into the destination register.
 - **Operation:** `rd <- (k & 0x000FFFFF) << 12`
- **Move**
 - **Syntax:** `mv <rd>, <rs>`
 - **Description:** Move the value from the source register to the destination register.
 - **Operation:** `rd <- rs`
- **Add Immediate**
 - **Syntax:** `addi <rd>, <rs1>, <k>`
 - **Description:** Add a 17-bit sign-extended immediate value to the source register and store the result in the destination register. The immediate `k` is truncated to the 17-bit `imm_sign` field and sign-extended to 32 bits before the addition.
 - **Operation:** `rd <- rs1 + signext(k[16:0])`
- **Add**
 - **Syntax:** `add <rd>, <rs1>, <rs2>`
 - **Description:** Add the values of two source registers and store the result in the destination register.
 - **Operation:** `rd <- rs1 + rs2`
- **Subtract**
 - **Syntax:** `sub <rd>, <rs1>, <rs2>`
 - **Description:** Subtract the value of the second source register from the first source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 - rs2`
- **Multiply**
 - **Syntax:** `mul <rd>, <rs1>, <rs2>`
 - **Description:** Multiply the values of two source registers and store the result in the destination register.
 - **Operation:** `rd <- rs1 * rs2`
- **Multiply High**

- **Syntax:** `mulh <rd>, <rs1>, <rs2>`
- **Description:** Multiply the values of two source registers and store the high part of the result in the destination register.
- **Operation:** `rd <- (rs1 * rs2) >> (word size)`
- **Divide**
 - **Syntax:** `div <rd>, <rs1>, <rs2>`
 - **Description:** Divide the value of the first source register by the value of the second source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 / rs2`
- **Remainder**
 - **Syntax:** `rem <rd>, <rs1>, <rs2>`
 - **Description:** Compute the remainder of the division of the first source register by the second source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 % rs2`
- **Logical Shift Left**
 - **Syntax:** `sll <rd>, <rs1>, <rs2>`
 - **Description:** Shift the value of the first source register left by the number of bits specified in the lower 5 bits of the second source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 << (rs2 & 0x1F)`
- **Logical Shift Right**
 - **Syntax:** `srl <rd>, <rs1>, <rs2>`
 - **Description:** Shift the value of the first source register right (zero-fill) by the number of bits specified in the lower 5 bits of the second source register and store the result in the destination register.
 - **Operation:** `rd <- rs1 >>> (rs2 & 0x1F)`
- **Arithmetic Shift Right**
 - **Syntax:** `sra <rd>, <rs1>, <rs2>`
 - **Description:** Shift the value of the first source register right by the number of bits specified in the lower 5 bits of the second source register, preserving the sign, and store the result in the destination register.
 - **Operation:** `rd <- rs1 >> (rs2 & 0x1F)`
- **Bitwise AND**
 - **Syntax:** `and <rd>, <rs1>, <rs2>`
 - **Description:** Perform a bitwise AND on the values of two source registers and store the result in the destination register.
 - **Operation:** `rd <- rs1 & rs2`
- **Bitwise OR**
 - **Syntax:** `or <rd>, <rs1>, <rs2>`

- **Description:** Perform a bitwise OR on the values of two source registers and store the result in the destination register.
- **Operation:** `rd <- rs1 | rs2`
- **Bitwise XOR**
 - **Syntax:** `xor <rd>, <rs1>, <rs2>`
 - **Description:** Perform a bitwise XOR on the values of two source registers and store the result in the destination register.
 - **Operation:** `rd <- rs1 ^ rs2`
- **Set Less Than Immediate**
 - **Syntax:** `slti <rd>, <rs1>, <k>`
 - **Description:** Set the destination register to 1 if the source register is less than the immediate (signed), else 0.
 - **Operation:** `rd <- (rs1 < k) ? 1 : 0`
- **NOP**
 - **Syntax:** `nop`
 - **Description:** No operation.

Slot 2: Memory Operations

- **Load Word**
 - **Syntax:** `lw <rd>, <offset>(<rs1>)`
 - **Description:** Load a word from memory at the address computed by adding the offset to the base register into the destination register.
 - **Operation:** `rd <- M[offset + rs1]`
- **Load Byte**
 - **Syntax:** `lb <rd>, <offset>(<rs1>)`
 - **Description:** Load a byte from memory at the address computed by adding the offset to the base register, sign-extend it to 32 bits, and store in the destination register.
 - **Operation:** `rd <- signext(M[offset + rs1][7:0])`
- **Store Word**
 - **Syntax:** `sw <rs2>, <offset>(<rs1>)`
 - **Description:** Store the value from the source register into memory at the address computed by adding the offset to the base register.
 - **Operation:** `M[offset + rs1] <- rs2`
- **Store Byte**
 - **Syntax:** `sb <rs2>, <offset>(<rs1>)`
 - **Description:** Store the lower 8 bits of the value from the source register into memory at the address computed by adding the offset to the base register.
 - **Operation:** `M[offset + rs1] <- rs2 & 0xFF`
- **NOP**

- **Syntax:** `nop`
- **Description:** No operation.

Slot 3: Control Operations

- **Jump**
 - **Syntax:** `j <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter.
 - **Operation:** `pc <- pc + k`
- **Jump and Link**
 - **Syntax:** `jal <rd>, <k>`
 - **Description:** Store the address of the next instruction in the destination register and jump to the address computed by adding the immediate value to the current program counter.
 - **Operation:** `rd <- pc + 14, pc <- pc + k // Adjusted for bundle size`
- **Jump Register**
 - **Syntax:** `jr <rs>`
 - **Description:** Jump to the address stored in the source register.
 - **Operation:** `pc <- rs`
- **Branch if Equal to Zero**
 - **Syntax:** `beqz <rs1>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the source register is zero.
 - **Operation:** `if rs1 == 0 then pc <- pc + k`
- **Branch if Not Equal to Zero**
 - **Syntax:** `bnez <rs1>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the source register is not zero.
 - **Operation:** `if rs1 != 0 then pc <- pc + k`
- **Branch if Greater Than**
 - **Syntax:** `bgt <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is greater than the value in the second source register.
 - **Operation:** `if rs1 > rs2 then pc <- pc + k`
- **Branch if Less Than or Equal**
 - **Syntax:** `ble <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is less than or equal to the value in the second source register.

- **Operation:** `if rs1 <= rs2 then pc <- pc + k`
- **Branch if Greater Than (Unsigned)**
 - **Syntax:** `bgtu <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the unsigned interpretation of the first source register is greater than the unsigned interpretation of the second source register.
 - **Operation:** `if unsigned(rs1) > unsigned(rs2) then pc <- pc + k`
- **Branch if Less Than or Equal (Unsigned)**
 - **Syntax:** `bleu <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the unsigned interpretation of the first source register is less than or equal to the unsigned interpretation of the second source register.
 - **Operation:** `if unsigned(rs1) <= unsigned(rs2) then pc <- pc + k`
- **Branch if Equal**
 - **Syntax:** `beq <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is equal to the value in the second source register.
 - **Operation:** `if rs1 == rs2 then pc <- pc + k`
- **Branch if Not Equal**
 - **Syntax:** `bne <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is not equal to the value in the second source register.
 - **Operation:** `if rs1 != rs2 then pc <- pc + k`
- **Branch if Less Than**
 - **Syntax:** `blt <rs1>, <rs2>, <k>`
 - **Description:** Jump to the address computed by adding the immediate value to the current program counter if the value in the first source register is less than the value in the second source register (signed).
 - **Operation:** `if rs1 < rs2 then pc <- pc + k`
- **NOP**
 - **Syntax:** `nop`
 - **Description:** No operation.
- **Halt**
 - **Syntax:** `halt`
 - **Description:** Halt the machine.

ISA Specific State Views

- `<reg>:dec` , `<reg>:hex` -- View the value of a specific register in decimal or hexadecimal format.

Available registers: Zero , Ra , Sp , Gp , Tp , T0 , T1 , T2 , S0Fp , S1 , A0 , A1 , A2 , A3 , A4 , A5 , A6 , A7 , S2 , S3 , S4 , S5 , S6 , S7 , S8 , S9 , S10 , S11 , T3 , T4 , T5 , T6 .

Slot utilization (parallelism)

Each VLIW bundle has four execution slots (memory, ALU1, ALU2, control). A slot containing the corresponding `nop` is idle; everything else counts as active. The simulator tallies how many slots each executed bundle used and exposes three summary view variables. They're typically used with `slice: last` because the values are run-totals.

- `vliw:load-percent` -- average slot utilization across the run, as an integer percent. $(\text{active_slots} * 100) / (\text{bundles} * 4)$. 25% means each bundle used one slot on average; 100% means every slot of every bundle was active.
- `vliw:avg-load` -- average number of active processing units (slots) per executed bundle, rendered with two decimals. $\text{active_slots} / \text{bundles}$, a value in 0.00 – 4.00 . This is the same measure as `vliw:load-percent` expressed as units-per-bundle instead of a percent of the four-wide width.
- `vliw:bundles-by-load` -- histogram rendered as comma-separated K:N (P%) entries, one per non-empty bucket. K is the active-slot count, N is the number of bundles in that bucket, P is its percent share of all executed bundles. Empty buckets are skipped. A peak at 1 means a serial program; a peak at 3-4 means the compiler / programmer is exploiting the pipeline.

Example -- print a load summary at the end of the simulation:

```
reports:
- name: vliw-load
  slice: last
  view: |
    vliw:load-percent:    {vliw:load-percent}
    vliw:avg-load:       {vliw:avg-load}
    vliw:bundles-by-load: {vliw:bundles-by-load}
```

For `hello.s` running on this ISA the summary reads:

```
vliw:load-percent:    81%
vliw:avg-load:        3.24
vliw:bundles-by-load: 1:3 (9%), 3:16 (48%), 4:14 (42%)
```

— 14 fully-packed bundles (4:14 (42%)), 16 three-wide (3:16 (48%)), 3 with a single active slot (1:3 (9%)), no idle or two-slot bundles, for an average of 3.24 active units per bundle.

All three lines are also emitted together by the generic `{isa-specific}` summary block, which lets a single report template stay uniform across ISAs.